MODULE 13 · DOMAIN 2 - APPLICATIONS AND INTEGRATION

Rate Limits, Errors and Platforms

Working within requests and tokens per minute, retrying the right errors the right way, and running Claude through Bedrock, Google Cloud, Foundry and Claude Platform on AWS.

DOMAIN 2 of 8 EXAM WEIGHT 33.1% MODULE 13 of 36

admin@sthithapragna.com

Rate limits are traffic lights on a motorway slip road. **Queue at the lights. Never floor it at a red.**

How Claude API rate limits work

FACT	DETAIL
What is measured	Requests, input tokens and output tokens per minute: RPM, ITPM, OTPM
Measured for	Each model class, at the organisation level; workspaces can be capped lower
Algorithm	Token bucket: capacity is continuously replenished, not reset each minute
Short intervals	60 RPM may be enforced as one per second, so bursts can fail
Sharp increases	Acceleration limits: ramp traffic up gradually
Tiers	Start, Build, Scale, Custom; new organisations may begin in Evaluation

ON THE EXAM

Under the limit on average is not under the limit. Bursts and sudden ramps fail too.

Two facts that change designs

- **Cache reads do not count toward ITPM** on most current models. Only uncached input and cache writes do.
- **Every response carries headers:** anthropic-ratelimit-...-remaining and -reset, and retry-after on a 429.
- **Read the headers** to slow down before you hit the limit, not after.
- **Foundry does not send these headers.** Platforms differ, so check yours.

> THE TAKEAWAY
>
> A large shared prefix, cached, costs less and frees input-token headroom at the same time.

Which errors to retry

ERROR	MEANING	ACTION
`429 rate_limit_error`	Over a limit	Wait retry-after, then back off
`429 spend cap`	Monthly cap reached; no retry-after	Stop. Alert. Retries fail until reset
`500, 529, connection`	Server error, overloaded, network	Retry with backoff and jitter
`504 timeout`	Request too slow	Retry; consider streaming
`400, 401, 403, 404, 413`	Your request or credentials	Never retry unchanged. Fix it

THE TIEBREAKER

A retry only helps if the cause is temporary. A malformed request fails the same way every time.

Retrying well

```python
def call_with_backoff(make_request, attempts=6):
    for attempt in range(attempts):
        try:
            return make_request()
        except anthropic.RateLimitError as e:
            if is_spend_cap(e): raise                # no retry-after: stop and alert
            wait = retry_after(e) or min(60, 2 ** attempt)
        except (anthropic.InternalServerError, anthropic.APIConnectionError):
            wait = min(60, 2 ** attempt)
        time.sleep(wait + random.uniform(0, 1))      # jitter spreads the retries
    raise GaveUp(attempts)
```

6-7 Spend cap stops; otherwise honour retry-after, then exponential.

10 Random jitter stops every worker retrying at the same instant.

CURRENCY CHECK

The SDKs already retry twice by default, honouring retry-after. Add your own layer for long jobs.

Designing within the limits

- **Queue and cap concurrency:** a worker pool sized to your limits, not to your data.
- **Smooth and ramp:** spread work over time; increase traffic gradually.
- **Protect interactive traffic:** background work in its own workspace, capped lower.
- **Spend fewer tokens:** cache shared prefixes, trim context, batch bulk work.
- **Need more?** Request a tier increase in the Console, with usage evidence.

> THE TIEBREAKER
>
> Raising limits is the last step, after the design stops wasting the ones you have.

Copperlet's 2am sweep, finished

BEFORE

A storm of 429s

- 310,000 synchronous calls at 2am
- 200 workers firing at once
- Immediate retries, no jitter
- Tenant assistant starved too

AFTER

Metered traffic

- Bulk notes in one batch (module 12)
- Stragglers: queue, 10 workers, backoff with jitter
- Own prod-background workspace, capped
- Spend-cap 429 pages the on-call engineer

SETTLED

Quirk 7 is now fully settled: fewer requests, metered, isolated and retried well.

Claude through other platforms

PLATFORM	CLIENT AND IDS	WATCH FOR
Amazon Bedrock	AnthropicBedrockMantle; IAM; anthropic.claude-...	No Batches, Files, server tools, Skills
Google Cloud	AnthropicVertex; model in the URL	Region by endpoint; no Batch
Microsoft Foundry	AnthropicFoundry; deployment name	No Batches; no rate-limit headers
Claude Platform on AWS	Run by Anthropic; AWS billing; same IDs	Near parity; no fast mode

ON THE EXAM

The model is the same. Features, IDs, limits, regions and authentication are not.

One thin seam, many platforms

THE SEAM

Copperlet's client module

- Platform and model ID from config
- Capability flags per platform
- One retry and logging policy
- Tests run against each platform

HARWOOD ON BEDROCK

Same features, different path

- Bedrock client, UK or EU region
- No Batches: sweep notes run queued
- No Files: documents sent inline
- Quotas set per AWS account

THE TAKEAWAY

Keep provider differences in one module. Business logic should never ask which cloud it is on.

> A nightly job starts 300 workers at once, each sending requests as fast as it can. Its average usage per minute is below the organisation's limits, yet it receives many 429 errors in the first few minutes.
>
> Workers retry immediately after each error.

What is the best fix?

A. Request a higher rate-limit tier before changing anything else

B. Queue the work with a concurrency cap, ramp up gradually, and retry with backoff and jitter

C. Move the job to a different model with the same prompt

D. Retry faster, so that each worker gets back into the queue sooner

◆ PAUSE AND DECIDE

What is the best fix?

A Request a higher rate-limit tier before changing anything else

The average already fits. Bursts are the problem.

B Queue the work with a concurrency cap, ramp up gradually, and retry with backoff and jitter

Removes the burst, the ramp and the retry storm.

C Move the job to a different model with the same prompt

Same burst, different bucket.

D Retry faster, so that each worker gets back into the queue sooner

Makes the storm worse.

THE SIGNAL

"Average below the limit" plus errors at the start: bursts and a sharp ramp. Meter the traffic.

> A team's client retries every failed Claude request ten times with a fixed one-second delay, whatever the error. Logs show malformed requests and invalid keys being retried, while overload errors still fail during busy periods.
>
> The team wants a sound retry policy.

Which policy is best?

A. Keep retrying every error, but increase the fixed delay to five seconds

B. Never retry, and surface every error straight to the user

C. Retry only bad-request and authentication errors, since those are the most common

D. Retry rate-limit, server, overload and connection errors with backoff and jitter; fix request and auth errors

◆ PAUSE AND DECIDE

Which policy is best?

A Keep retrying every error, but increase the fixed delay to five seconds

Still retries errors that can never succeed.

B Never retry, and surface every error straight to the user

Fails on errors that would clear in seconds.

C Retry only bad-request and authentication errors, since those are the most common

Exactly backwards.

D Retry rate-limit, server, overload and connection errors with backoff and jitter; fix request and auth errors

Retry the temporary, fix the permanent.

THE SIGNAL

Retry what is temporary, with growing, jittered waits. Fix what is permanent.

> Mid-month, every request from a company's production app starts returning 429. The error details say enforced_spend_limit_reached, and there is no retry-after header.
>
> The app's retry loop has been running for hours.

What should the team do?

A Stop the retries, alert the owners, and raise the spend cap or wait for the reset

B Lengthen the backoff, so the retries eventually succeed

C Create a new API key in the same organisation and retry with it

D Move the traffic to another workspace in the same organisation

◆ PAUSE AND DECIDE

What should the team do?

A. Stop the retries, alert the owners, and raise the spend cap or wait for the reset
Retries fail until access resumes.

B. Lengthen the backoff, so the retries eventually succeed
They will not, until the cap changes.

C. Create a new API key in the same organisation and retry with it
The cap is on the organisation, not the key.

D. Move the traffic to another workspace in the same organisation
Organisation limits apply across workspaces.

CURRENCY CHECK

A 429 without retry-after, marked spend limit, is not a rate limit to wait out. It is a stop sign.

A customer assistant sends the same 40,000-token system prompt and policy documents with every request. At peak hours it hits its input-tokens-per-minute limit. A nightly report job runs real-time requests against the same limits.

The team wants more headroom without a tier change.

Which two changes help most? Choose two.

A. Raise max_tokens so that each response does more work

B. Cache the shared prefix, since cache reads do not count toward ITPM on current models

C. Send the policy documents in the user turn instead of the system prompt

D. Switch the assistant from streaming to non-streaming responses

E. Move the nightly report job to the Message Batches API

◆ PAUSE AND DECIDE

Which two changes help most? Choose two.

A Raise max_tokens so that each response does more work

Output, not input. No headroom gained.

B Cache the shared prefix, since cache reads do not count toward ITPM on current models

The repeated 40,000 tokens stop counting.

C Send the policy documents in the user turn instead of the system prompt

Same tokens, different place.

D Switch the assistant from streaming to non-streaming responses

Streaming does not change token counts.

E Move the nightly report job to the Message Batches API

Takes bulk work off the real-time limits.

THE SIGNAL

Repeated prefix: cache it. Background work competing for limits: move it to Batch.

> A company moves one enterprise customer's traffic to Amazon Bedrock, reusing code written for the first-party Claude API. Calls that use the Message Batches API and the Files API fail on Bedrock.
>
> Other customers stay on the first-party API.

What is the best approach?

A Check the credentials, since Bedrock supports every first-party feature through the same client

B Fork the whole application into a separate Bedrock version

C Check feature availability per platform, and use capability flags with a fallback path behind the client seam

D Ask Claude in the prompt to process the requests as a batch

◆ PAUSE AND DECIDE

What is the best approach?

A Check the credentials, since Bedrock supports every first-party feature through the same client

It does not, and it uses a different client.

B Fork the whole application into a separate Bedrock version

Two codebases to keep in step forever.

C Check feature availability per platform, and use capability flags with a fallback path behind the client seam

Same features, different path, one codebase.

D Ask Claude in the prompt to process the requests as a batch

Batching is an API feature, not a prompt.

THE SIGNAL

Features differ by platform. Check the platform's page, then route around gaps in one place.

> A software company must serve some customers through the first-party Claude API and others through Google Cloud. Model IDs and client setup are currently hard-coded in dozens of files.
>
> More platforms may follow.

What is the best design?

A Copy the codebase once per platform, and maintain each copy separately

B Adopt a large multi-vendor agent framework purely to switch providers

C Add if-statements for each platform wherever a model ID appears

D One internal client module that takes platform and model ID from configuration, tested on each platform

◆ PAUSE AND DECIDE

What is the best design?

A Copy the codebase once per platform, and maintain each copy separately

Every fix made twice, or forgotten once.

B Adopt a large multi-vendor agent framework purely to switch providers

A heavy dependency for a thin problem.

C Add if-statements for each platform wherever a model ID appears

Spreads the problem across dozens of files.

D One internal client module that takes platform and model ID from configuration, tested on each platform

One thin seam, configured, tested.

THE SIGNAL

Hard-coded IDs in dozens of files: centralise behind one thin, configured seam.

Module 13 in five lines

- **RPM, ITPM, OTPM per model class;** token bucket; bursts and sharp ramps fail too.
- **Cache reads do not count toward ITPM;** headers tell you what remains.
- **Retry the temporary** with backoff, jitter and retry-after; fix the permanent.
- **Spend-cap 429:** no retry-after, so stop and alert.
- **Platforms differ in features:** one thin seam, flags and fallbacks.

COMING UP

Next: JSON, async and concurrency in Claude applications.